Docs Chain Wiring Design — helix_track

Revision: 1 **Last modified:** 2026-06-17T05:01:56Z **Status:** DESIGN + SCAFFOLD (background work unit). Registers the project's docs_chain contexts and specifies the bidirectional SQL-definitions ⍰ documentation sync mandated by the operator (2026-06-17). Does NOT yet run `sync`/`verify` — see §6 prerequisites. **Authority:** Operator mandate 2026-06-17 (SQL-definitions ⍰ documentation bidirectional sync via docs_chain) + Constitution §11.4.106 (Docs Chain). **Scope:** root `helix_track` monorepo. Contexts live at `.docs_chain/contexts/`. The engine is the `docs_chain` submodule at repo root, inherited BY REFERENCE — never copied.

---

1. What this delivers

Two registered docs_chain contexts plus the design behind them:

Context file	Purpose	Status
`.docs_chain/contexts/sql_definitions.yaml`	Bidirectional sync of Core's SQL DDL ⍰ System SQL-reference doc	**DRAFT** (blocked on rename + exec transforms — §6)
`.docs_chain/contexts/tracker_docs.yaml`	CONTINUATION.md → html → pdf export sync (§12.10/§11.4.65)	**READY** (real nodes; syncs once the binary is built)

The contexts are written against the **live, IMPLEMENTED** docs_chain config contract (`docs_chain/docs/CONFIG_SCHEMA.md` Revision 3, Phase-4 loader). The schema fields used are exactly those the loader validates; no invented fields.

---

2. The real docs_chain schema (captured, not assumed)

Source of truth read this session: `docs_chain/docs/CONFIG_SCHEMA.md` (Rev 3, "IMPLEMENTED — parsed + validated by the Phase-4 config loader `internal/config`") and `docs_chain/docs/USE_CASE_CATALOGUE.md` (Rev 3).

A context YAML (one file per context, under `.docs_chain/contexts/<name>.yaml`, stem SHOULD match `context:`):

```
context: <string>          # REQUIRED, unique, matches filename
        stem
description: <string>      # OPTIONAL
nodes: <map>               # REQUIRED: node_id -> { kind, path[,
        members, exclude] }
edges: <list>              # REQUIRED: derive-from and/or sync
        edges
transforms: <map>          # REQUIRED if any edge names a
        transform
```

Node kinds (closed set): `markdown`, `html`, `pdf`, `sqlite`, `summary`, `status`, `status_summary`, `fingerprint` (the last REQUIRES a `members:` glob; optional `exclude:` list).

Edge shapes: - `derive-from` (one-way): `{ type: derive-from, from: <id|[ids]>, to: <id>, transform: <name> }` — `from` may be a list (multi-input transform). - `sync` (bidirectional): `{ type: sync, a: <id>, b: <id>, authority: <a|b>, transform_a_to_b: <name>, transform_b_to_a: <name> }` — `authority` MUST equal `a` or `b`; transforms may be omitted only when the `(kind_a, kind_b)` pair maps to a known builtin pair.

Transform spec: exactly one of `{ builtin: <name> }` OR `{ exec: <cmd>, args: [...] }`. Builtins: `pandoc-html`, `weasyprint-pdf`, `colorize-html`, `gen-summary`, `md-to-sqlite`, `sqlite-to-md`, `members-fingerprint`. `exec:` contract (§5.2): Docs Chain stages input(s) to temp file(s), passes the staged input temp path(s) then the staged output temp path; `args:` entries are appended **after** those; the script MUST read only declared inputs, write only to the staged output temp, exit 0/non-zero, and be **byte-stable** (§11.4.50).

CLI / exit contract: `docs_chain sync|verify|doctor|graph [<context>|--all]`. `doctor` validates contexts; `sync` propagates; `verify` is the deterministic sink-side gate (exit 0 = in sync). Both-dirty `sync` → conflict (exit 2, no writes). Validation failure → exit 4. Transform failure → rollback, exit 3.

`state.json` + `*.docs_chain.tmp` live under `.docs_chain/` and MUST be gitignored (§11.4.77 regen mechanism = `docs_chain sync`).

---

3. Inherited-by-reference invocation path (§11.4.106 / §11.4.28 / §11.4.80)

The engine is the `docs_chain` submodule at repo root. It is consumed BY REFERENCE — the consumer NEVER copies the Go engine into the project tree (same pattern §11.4.80 uses for `codegraph_*`).

Build once (the submodule owns its build per `docs_chain/CLAUDE.md`):

```
# from repo root — builds the engine in-place inside the submodule
( cd docs_chain && go build -o ./docs_chain ./cmd/docs_chain )
```

Invoke against this project's contexts (run from repo root so node `path:` values resolve project-root-relative):

```
./docs_chain/docs_chain doctor   --all   # validate every context
./docs_chain/docs_chain sync     --all   # propagate
./docs_chain/docs_chain verify   --all   # deterministic in-sync
        gate (pre-build / CI)
```

The contexts (`.docs_chain/contexts/*.yaml`) are project-owned DATA the consumer registers (§11.4.28-B decoupling); the engine stays project-agnostic. A future Phase-6 step (operator-gated, §11.4.66) would expose the engine via the constitution submodule so the bare `docs_chain` command is on PATH; until then the `./docs_chain/docs_chain` path is authoritative.

---

4. SQL definitions ⁉ documentation (the operator mandate, 2026-06-17)

4.1 Inventory (git index only — safe during the live Core→core rename)

Read via `git -C Core ls-files 'Database/DDL/*.sql' 'Database/**/*.sql'` (the Core submodule's own index; the parent `git ls-files` shows only the gitlink). **21 SQL DDL files** inventoried:

- Core schema: `Definition.V1..V5.sql` (5)
- Core migrations: `Migration.V1.2 / V2.3 / V3.4 / V5.6.sql` (4)
- `Indexes_Performance.sql`, `Seed_Data.sql` (2)
- Extensions/Chats: `Definition.V1`, `Definition.V2`, `Migration.V1.2` (3)
- Extensions/Documents: `Definition.V1`, `Definition.V2`, `Migration.V1.2` (3)
- Extensions/Times: `Definition.V1` (1)
- Services/Authentication: `Definition.V1` (1)
- Services/Localization: `Definition.V1`, `Migration.V1.2` (2)

(`Database/Test/Test.Postgres.sql` is a test fixture, not a schema definition — excluded from the definitions chain by design.)

4.2 Path resolution (snake_case `core/`)

The `Core` submodule gitlink has been renamed to `core` (snake_case migration §11.4.29, landed). docs_chain node `path:` values are **project-root-relative**; a submodule's internal files are reachable from the parent root at `<gitlink>/Database/DDL/...`. The context therefore references the current path `core/Database/DDL/...` — these resolve now that the rename has landed.

4.3 Bidirectional modelling

The mandate is "bidirectionally synced." docs_chain's `sync` edge is **pairwise (a ? b)**. With 21 SQL sources and one documentation surface, two faithful modellings exist:

- **(M1) multi-input `derive-from` (sql set → doc)** — supported today (`from:` may be a list); regenerates the System SQL-reference doc from the DDL set. This is the SQL→doc direction only.
- **(M2) per-file `sync` edges (each `.sql` ? a doc section)** — true bidirectional, but requires either one doc node per `.sql` or a section-addressable doc, plus a transform pair per file.

The DRAFT context ships **M1** (the safe, implementable-first half) and flags M2 as the operator-decision (§4.4 Q1). Reason: making the human-readable doc *authoritative for schema* (doc→sql writing real DDL) is high-blast-radius (§11.4.133 target-system safety — generated DDL touches the database) and MUST be operator-confirmed before wiring.

4.4 The transform pair (NOT a builtin — must be authored)

docs_chain builtins `md-to-sqlite` / `sqlite-to-md` operate on **tabular pipe-table markdown ? a sqlite DB**. They do NOT parse DDL `CREATE TABLE` statements. The SQL-definition ? doc sync therefore needs project-owned `exec:` transforms:

- `scripts/dc/sql_to_doc.sh` — render the DDL set into the System SQL-reference Markdown (table/column inventory, version diffs, FK graph). **NOT YET IMPLEMENTED.** Must be byte-stable (§11.4.50) and 100%-test-covered (§11.4.27).
- `scripts/dc/doc_to_sql.sh` (only if Q1 approves doc-authoritative) — project doc edits back into DDL. **Deliberately unwired in the DRAFT.**

These scripts do not exist yet; the context references them as placeholders so the shape is reviewable. Per §11.4.106, NO faked transform is permitted — the context stays `doctor`-valid but `sync`-inert until the scripts land.

---

5. tracker_docs context

`docs/CONTINUATION.md` is the only tracker/handoff doc present at the repo root (verified via `git ls-files docs/` — Issues.md / Fixed.md are ABSENT). The context wires `CONTINUATION.md → .html → .pdf` (§12.10 + §11.4.65) using the `pandoc-html` / `weasyprint-pdf` builtins — REAL and runnable as soon as the binary is built. Issues/Fixed nodes are intentionally omitted (registering absent sources would fail loader validation rule 2). When the §11.4.93 tracker constellation is adopted at the root, register it via USE_CASE_CATALOGUE recipes (a)+(b).

---

6. Prerequisites before `sync`/`verify` can run

1. **Build the engine** — `( cd docs_chain && go build -o ./docs_chain ./cmd/docs_chain )` (the submodule's acceptance test must be GREEN).
2. **Core→core rename landed** — `sql_definitions.yaml` paths resolve only after the gitlink is named `core` and the submodule is checked out.
3. **Author the `exec:` transforms** — `scripts/dc/sql_to_doc.sh` (and `doc_to_sql.sh` if Q1 approves) with 100% test coverage (§11.4.27) + byte-stability (§11.4.50).
4. **Author the seed `docs/database/SQL_REFERENCE.md`** with a §11.4.44 revision header (the source author owns the header; docs_chain only keeps exports in sync).
5. **`.gitignore`** — add `.docs_chain/state.json` and `.docs_chain/*.docs_chain.tmp` (operator/main-stream edits the root `.gitignore`; this unit must not).
6. **CI / pre-build wiring** — `docs_chain verify --all` as a gate (operator-gated where it edits a gate file).

This background unit performed steps 0 only (design + context scaffold). Steps 1–6 are tracked follow-ups; none were faked (§11.4.6 / §11.4.106).

---

7. Open questions for the operator (§11.4.66)

- **Q1 — SQL-definition authority direction.** Is the **`.sql` DDL the sole authority** (doc is always derived, M1 — recommended, lowest blast radius), or may the **System SQL-reference doc be authoritative** for schema (doc→sql generates real DDL, M2 — requires the `doc_to_sql.sh` transform + a §11.4.133 safety review since generated DDL touches the database)?
- **Q2 — Multi-source authority node.** docs_chain `sync` `authority:` is a single node. If M2 (per-file `sync`) is chosen, each `.sql` syncs with its own doc section; confirm the doc is section-addressable, or accept one doc file per `.sql` (21 docs).
- **Q3 — Doc granularity.** One consolidated `docs/database/SQL_REFERENCE.md` (current DRAFT) vs per-schema-version / per-extension docs? Affects node count and the `sql_to_doc.sh` output contract.
- **Q4 — Post-rename path confirmation.** Confirm the renamed gitlink is exactly `core` (not `core_service` / other) so the `core/Database/DDL/...` paths are correct.

---

8. Files created by this unit

- `.docs_chain/contexts/sql_definitions.yaml` (DRAFT)
- `.docs_chain/contexts/tracker_docs.yaml` (READY)
- `docs/docs_chain/WIRING_DESIGN.md` (this file)

No `git add`/commit/push performed. No `.gitmodules` edit. No docs_chain submodule tracked-file modified. No working directory of a renaming submodule touched (SQL inventory read via `git -C Core ls-files`, index-only).